JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 202219459 이름: 한승우 제출일: 10/27

서비스 이름 학습 기록 게시판

기술 조합과 버전 React 19.3.0 (Vite 8.3.4) / Spring Boot 3.5.16 (JJWT 0.13.0, SQLite) / Windows 11, Node.js v24.21.0, JDK 21.0.12.1

GitHub 저장소 URL: https://github.com/hanseungwoo0614/assignment

소스 코드 기준 main 브랜치 / 코드 구현 기준 커밋 0912c2d (이후 커밋은 README 수정)

오늘의 학습 기록 게시판은 같은 수업을 듣는 학생들이 공부한 내용을 짧게 기록하고 서로의 학습 과정을 함께 읽는 게시판이다. 기록은 로그인 없이 누구나 읽을 수 있지만, 글 작성은 로그인한 사용자만 할 수 있고 수정·삭제는 작성자 본인만 할 수 있다.

기술 조합은 React 와 Spring Boot 를 선택했다. Spring Boot 는 수업 실습에서 사용해와서 이해하기 쉬웠고, IntelliJ 에서 코드 수정을 이어나아갔다.

제공 템플릿에서 변경한 범위는 다음과 같다. 백엔드는 AuthService.java 의 생성자, issueToken, currentUserId, assertOwner 를 구현하고 bearerToken 함수를 추가했다. 프론트는 api.js 의 authorizationHeaders, request 의 오류 처리, login, canEdit 을 구현했고, main.jsx 는 서비스 제목과 안내 배너 문구만 바꿨다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	BoardController.list, get	토큰 없이 200 조회
로그인과 JWT 발급	완료	AuthService, issueToken / api.js login	TTL 기본 90 초
인증과 내 정보 조회	완료	AuthService.currentUserId, bearerToken / BoardController.me	토큰 없음·변조·만료 401 구분
본인 글 작성·수정·삭제	완료	AuthService.assertOwner / api.js canEdit, authorizationHeaders	타인 글 변경 403, 작성자 위조 400
만료·로그아웃·오류 표시	완료	api.js request, clearSession	404 일 때 안내만 하고 목록 자동 갱신은 하지 않음
DB 데이터 유지	완료	PostStore, DatabaseConfig	backend-spring/data/board.sqlite, 재시작 후 유지 확인

구조와 변경 내역

전체 구조

```
브라우저 (React 빌드본, http://127.0.0.1:4173)
   │ /api 로 시작하는 요청
   ▼
Vite preview 프록시 (vite.config.js 의 preview.proxy)
   │ http://127.0.0.1:8000 으로 전달
   ▼
Spring Boot API 서버 (BoardController → AuthService → PostStore)
   │ JDBC
   ▼
SQLite 파일 DB (backend-spring/data/board.sqlite)
```

토큰은 브라우저 탭의 메모리(api.js 의 session 변수)에만 저장한다. localStorage·sessionStorage 는 사용하지 않으므로 새로고침하면 로그인이 해제된다. 게시글은 서버의 SQLite 파일에 저장되므로 새로고침이나 서버 재시작 후에도 유지된다.

흐름 1: 로그인 요청

```
① 화면 로그인 버튼 (main.jsx)
② api.js login() → POST /api/auth/login {email, password}  ※ Authorization 헤더 없음
③ BoardController.login()
     fields(): 입력 형식 검사 → 실패 시 400 BAD_INPUT
     Users.verify(): 비밀번호 해시 비교 → 실패 시 401 INVALID_CREDENTIALS
④ AuthService.issueToken(): HS256 JWT 발급 (sub, iss, aud, iat, exp)
⑤ 200 응답 {accessToken, expiresIn, user}
⑥ api.js 가 토큰과 user 를 session(메모리)에 저장 → 화면에 "Alice 님 로그인 중"
```

흐름 2: 게시글 수정 요청

```
① 수정 저장 버튼 (본인 글일 때만 canEdit()이 true 라 버튼이 보임)
② api.js update() → PUT /api/posts/{id}
     authorizationHeaders()가 Authorization: Bearer <JWT> 첨부  ※ 토큰 전송 지점
③ BoardController.update() 검사 순서
     1) AuthService.currentUserId(): 헤더·서명·iss·aud·만료·sub·계정 → 실패 시 401
     2) postId(): ID 형식 → 400 BAD_INPUT
     3) existing(): 글 존재 → 404 NOT_FOUND
     4) AuthService.assertOwner(): 작성자 = 현재 사용자? → 아니면 403 FORBIDDEN
     5) fields(): 제목·내용 검사 → 400 BAD_INPUT
     6) PostStore.update(): DB 변경 (작성자·createdAt 유지)
④ 200 → 화면 갱신. 401 이면 session 을 비워 로그아웃, 403 이면 로그인 유지
```

토큰을 전송하는 요청은 GET /api/me, POST /api/posts, PUT·DELETE /api/posts/{id}이다. 공개 요청(GET /api/posts, GET /api/posts/{id})은 서버가 토큰을 확인하지 않는다. 로그인 요청은 login()이 먼저 session 을 비우므로 헤더 없이 전송된다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend-spring/.../AuthService.java / 생성자	없음	JWT_SECRET 이 비었거나 예시 키거나 64 자리 16 진수가 아니면 서버 시작 거부. 32 바이트 키로 변환하고 iss·aud·TTL·Users 주입	BE-01-2: 약한 키로 토큰을 위조하지 못하게

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
AuthService.issueToken	501 NOT_IMPLEMENTED 반환	HS256 으로 sub·iss·aud·iat·exp 를 넣은 JWT 발급	BE-01-3
AuthService.currentUserId	항상 1 반환	Bearer 추출 → JJWT 로 서명·iss·aud·만료 검증 → HS256·exp·sub 형식·계정 존재 확인 → 검증된 ID 반환. 실패 시 401 code 구분	BE-02: 데모 사용자 고정 제거
AuthService.bearerToken (추가)	없음	Authorization 헤더가 없거나 'Bearer <토큰>' 형식이 아니면 401 TOKEN_REQUIRED	BE-02-2
AuthService.assertOwner	검사하지 않음	글 작성자와 현재 사용자 ID 가 다르면 403 FORBIDDEN	BE-03
frontend-react/src/api.js / login	오류를 던짐	POST /api/auth/login 호출 후 토큰과 user 를 메모리에 저장	FE-01
api.js / authorizationHeaders	빈 객체 반환	토큰이 있을 때만 Authorization: Bearer 헤더 생성	FE-02
api.js / request	204 외 무조건 JSON 파싱, 401 처리 없음	본문을 안전하게 파싱, 401 이면 session 삭제, code 별 한글 안내	FE-03
api.js / canEdit	항상 true	로그인 사용자 id 와 글 authorId 비교	FE-04
frontend-react/src/main.jsx / 제목·배너 문구	시작 코드 안내 문구	서비스 이름과 이용 안내로 교체	FE-04: 데모 배너 교체

작성 출처: AuthService.java 와 api.js 의 위 함수들은 AI 가 작성한 초안을 바탕으로 추가적으로 작성했다.

로컬 실행 기록

환경: Windows 11 / Node.js v24.21.0 / JDK 21.0.12.1 IntelliJ 터미널에서 백엔드 실행, 시스템 설치 JDK 는 25.0.4.1

환경설정: PowerShell 에서 $env:TOKEN_TTL='90'으로 설정했다. Spring 은 .env 를 읽지 않는다고 터미널 환경변수로 주입했다.

백엔드: backend-spring 폴더에서 인텔리제이의 위 실행버튼으로 'Started BoardApplication' 8000 번 포트 실행을 확인했다.

프론트: frontend-react 폴더에서 npm ci → npm run build → npm run preview

SQLite: backend-spring/data/board.sqlite (서버 시작 시 자동 생성)

4173 접속: http://127.0.0.1:4173 에서 Alice 로그인과 글 등록을 수행했고, Network 에서 POST /api/posts 201 과 Authorization: Bearer 헤더를 확인했다.

재시작 유지: 글 작성 후 백엔드를 Ctrl+C 로 종료하고 같은 명령으로 재실행한 뒤 새로고침하자 글·작성자·내용이 그대로 남아 있었다.

증거 위치: docs/evidence/

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: backend-spring/src/main/java/edu/classroom/board/BoardController.java / login (63~68 줄, 제공 코드), AuthService.java / issueToken (56~67 줄)

코드 발췌

```
// BoardController.login
var input = fields(body, Map.of("email", 254, "password", 128));
var user = users.verify(input.get("email"), input.get("password"));
if (user == null) throw new ApiError(401, "INVALID_CREDENTIALS", "...");
return Map.of("accessToken", auth.issueToken(user), "expiresIn", ttl, "user", user);

// AuthService.issueToken
Instant now = Instant.now();
return Jwts.builder()
    .subject(String.valueOf(user.id()))
    .issuer(issuer)
    .audience().add(audience).and()
    .issuedAt(Date.from(now))
    .expiration(Date.from(now.plusSeconds(ttlSeconds)))
    .signWith(key, Jwts.SIG.HS256)
    .compact();
```

동작과 응답 코드 설명

입력: POST /api/auth/login 의 JSON {email, password}.

처리: 제공 코드 fields()가 email·password 가 문자열인지와 다른 필드가 없는지 검사한다. 비밀번호는 앞뒤 공백을 지우지 않고 그대로 비교한다. Users.verify()는 저장된 해시와 비교하고, 없는 이메일도 같은 계산을 거쳐 응답 차이로 계정 존재를 알 수 없게 한다. 통과한 사용자만 issueToken()에 들어간다. issueToken()은 sub(사용자 ID 문자열 "1"/"2"), iss=webservice-class, aud=board-api, iat, exp(발급 시각+TTL, 기본 90 초)를 넣고 생성자에서 64 자리 16 진수를 32 바이트로 바꾼 키로 서명한다. 비밀번호와 해시는 토큰에 넣지 않는다.

출력: 200 {accessToken, expiresIn, user{id,name,email}}.

실패: 형식 오류·추가 필드·깨진 JSON → 400 BAD_INPUT, 틀린 비밀번호·없는 계정 → 401 INVALID_CREDENTIALS(토큰 미발급). JWT_SECRET 이 비었거나 예시 키거나 형식이 틀리면 서버가 시작되지 않는다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend-react/src/api.js / authorizationHeaders , request , login

코드 발췌

```
function authorizationHeaders() {
  if (!session.accessToken) return {};
  return { Authorization: `Bearer ${session.accessToken}` };
}
// request() 안 실패 처리
```

```
if (!response.ok) {
  const code = data?.code;
  if (response.status === 401) clearSession();
  let message = ERROR_MESSAGES[code];
  ...
}
export async function login(email, password) {
  clearSession();
  const data = await request("/auth/login", {
    method: "POST", body: JSON.stringify({ email, password }) });
  session = { accessToken: data.accessToken, user: data.user };
  return data.user;
}
```

동작과 응답 코드 설명

입력: 로그인 응답, 이후 각 API 호출.

처리: login()은 먼저 clearSession()으로 이전 상태를 비우고, 응답의 accessToken 과 user 를 모듈 변수 session 에 저장한다. localStorage·sessionStorage 는 쓰지 않아 새로고침하면 사라진다. authorizationHeaders()는 토큰이 있을 때만 Authorization: Bearer <JWT>를 만들고, 없으면 빈 객체를 돌려 'Bearer undefined' 같은 헤더가 생기지 않는다. request()는 204 면 본문을 읽지 않는다.

출력: 성공 시 응답 데이터를 반환하고 main.jsx 의 run()이 getSession().user 로 화면 로그인 표시를 갱신한다.

실패: 401 이면 clearSession()으로 토큰·사용자를 지워 로그아웃 상태가 되고 코드별 안내를 띄운다. 403 FORBIDDEN 은 로그인 상태를 유지하고 권한 안내만 한다. 400 은 서버 message 를 함께 보여 주고 입력값을 유지한다. 로그아웃 버튼은 clearSession()과 setUser(null)로 메모리와 화면을 즉시 지운다.

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: backend-spring/src/main/java/edu/classroom/board/AuthService.java / currentUserId, bearerToken

코드 발췌

```
String token = bearerToken(request.getHeader("Authorization"));
Jws<Claims> jws;
try {
  jws = Jwts.parser().verifyWith(key)
      .requireIssuer(issuer).requireAudience(audience)
      .build().parseSignedClaims(token);
} catch (ExpiredJwtException e) {
  throw new ApiError(401, "TOKEN_EXPIRED", "...");
} catch (JwtException | IllegalArgumentException e) {
  throw new ApiError(401, "INVALID_TOKEN", "...");
}
if (!"HS256".equals(jws.getHeader().getAlgorithm())) throw ... INVALID_TOKEN
Claims claims = jws.getPayload();
if (claims.getExpiration() == null) throw ... INVALID_TOKEN
String sub = claims.getSubject();
if (sub == null || !sub.matches("[1-9][0-9]{0,15}")) throw ... INVALID_TOKEN
long userId = Long.parseLong(sub);
if (users.find(userId) == null) throw ... INVALID_TOKEN
return userId;
```

동작과 응답 코드 설명

입력: 보호 API 요청의 Authorization 헤더.

처리: bearerToken()이 헤더를 공백으로 나눠 정확히 'Bearer'+토큰 두 조각인지 확인한다. 그다음 JJWT 의 verifyWith(key)로 서명을 다시 계산해 비교하고, requireIssuer·requireAudience 로 iss·aud 를 확인하며 exp 가 있으면 만료도 검사한다. 이후 알고리즘이 HS256 인지, exp 가 존재하는지(JJWT 는 exp 가 없으면 만료 검사를 건너뛰므로 직접 확인), sub 가 양의 정수 문자열인지, 그 ID 의 계정이 실제로 있는지 확인한다.

출력: 모든 검사를 통과한 사용자 ID 만 반환하고, 컨트롤러는 이 ID 를 /me 조회와 글 작성·수정·삭제에 사용한다. /me 는 200 .

실패: 헤더 없음·Bearer 형식 오류 → 401. 서명은 맞고 만료만 지난 토큰 → 401 (JJWT 가 서명을 먼저 확인하므로 ExpiredJwtException 은 서명이 정상일 때만 발생하고, JwtException 의 하위 클래스라 먼저 catch 한다). 변조·서명 불일치·JWT 형식 아님·iss/aud 불일치·HS256 아님·exp 없음·sub 오류·없는 사용자 → 401 INVALID_TOKEN. 모든 401 에는 제공 코드 Errors.api 가 WWW-Authenticate: Bearer 헤더를 붙인다.

3 4 작성자 권한 검사

파일 / 함수: AuthService.java / assertOwner , BoardController.java / create, update

코드 발췌

```
// AuthService.assertOwner
```

```
public void assertOwner(long userId, long authorId) {
  if (userId != authorId) {
    throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정·삭제할 수 있습니다.");
  }
}
// BoardController.update
long user = auth.currentUserId(req), pid = postId(id);   // 401, 400
var p = existing(pid);                                   // 404
auth.assertOwner(user, p.authorId());                    // 403
var input = fields(body, Map.of("title", 100, "content", 5000)); // 400
return store.update(pid, input.get("title"), input.get("content"));
// BoardController.create
long user = auth.currentUserId(req);
var p = store.create(input.get("title"), input.get("content"), user);
```

동작과 응답 코드 설명

비교 대상: currentUserId()가 토큰을 검증해 돌려준 현재 사용자 ID 와, DB 에 저장된 글의 authorId 를 비교한다.

처리: BoardController.update/delete 는 인증(401) → ID 형식(400) → 글 존재(404) → assertOwner(403) → 입력 검사(400) → DB 변경 순서로 실행된다. assertOwner 에서 예외가 나면 그 아래 store.update/delete 까지 가지 않으므로 DB 가 바뀌지 않는다. PostStore.update 는 title·content·updated_at 만 바꾸므로 작성자와 createdAt 은 유지된다.

401 과 403: 401 은 누구인지 확인 못 함이라 다시 로그인해야 하고, 403 은 로그인은 정상이지만 본인 글이 아님이라 로그인을 유지한 채 거부한다. 그래서 Bob 이 Alice 글에 공백 제목을 보내면 입력 검사보다 권한 검사가 먼저라 400 이 아니라 403 이 나온다.

작성자 위조 방지: 글 작성 시 작성자는 요청 본문이 아니라 currentUserId()의 결과로 정해진다(store.create(..., user)). 본문에 authorId·userId 같은 추가 필드가 있으면 fields()가 400 BAD_INPUT 으로 거부한다. 화면의 canEdit()은 버튼 표시용이고 실제 차단은 서버가 한다.

응답: 본인 수정 200, 본인 삭제 204(빈 본문), 타인 수정·삭제 403 FORBIDDEN.

테스트 결과

실행 환경: Windows 11 / Node.js v24.21.0, JDK 21.0.12.1 / Chrome / 백엔드 java -jar target/jwt-board-1.0.0.jar, 프론트 npm run preview, 자동 검사 py tools/check_api.py --mode final --expiry

로컬 배포 확인: npm run build 성공 → npm run preview(4173)에서 로그인·글 등록 API 연결 확인 / 백엔드 .\mvnw.cmd package 후 java -jar 실행 /

테스트 일시 및 코드 버전: 2026-10-17 /

토큰 TTL: 일반 90 초 / 만료 테스트 3 초

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	비로그인으로 GET /api/posts, GET /api/posts/{id}	200	200 / 해당 없음 (public list, public detail)	PASS	08-API-전체-테스트-통과.png
T02	Alice 로그인 (4173 화면 + API)	200, JWT·user, 화면 표시	200 / 해당 없음, accessToken 3 조각·user{id,name,email}, 화면 'Alice 님 로그인 중'	PASS	03-로그인-응답.png, 11-프론트엔드-화면.png, 13-글등록-Bearer-헤더.png
T03	alice 틀린 비밀번호 / nobody@class.test 로그인	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS (2 건), 토큰 없음	PASS	05-로그인-입력값-검증.png, 08-API-전체-테스트-통과.png
T04	토큰 없이 GET /api/me, POST /api/posts	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED (2 건), WWW-Authenticate: Bearer	PASS	06-인증-헤더-검증.png, 08-API-전체-테스트-통과.png
T05	Alice 토큰으로 글 작성 후 /me	201 authorId 1, /me Alice	201 / 해당 없음, Location 일치, authorId 1 / /me 200 id 1	PASS	07-게시글-작성자-userId-조작-차단.png, 13-글등록-Bearer-헤더.png, 08-API-전체-테스트-통과.png
T06	Alice 토큰으로 본인 글 수정·삭제	200, 204	200 (작성자·createdAt 유지) / 204, 이후 조회 404	PASS	08-API-전체-테스트-통과.png
T07	Bob 토큰으로 Alice 글 PUT	403 FORBIDDEN, 원문 유지	403 / FORBIDDEN, 재조회 결과 동일	PASS	08-API-전체-테스트-통과.png
T08	Bob 토큰으로 Alice 글 DELETE	403 FORBIDDEN, 글 유지	403 / FORBIDDEN, 재조회 결과 동일	PASS	08-API-전체-테스트-통과.png
T09	서명 첫 글자를 바꾼 토큰으로 GET /me, POST, PUT, DELETE	401 INVALID_TOKEN, 변경 없음	401 / INVALID_TOKEN (4 건), 재조회 결과 동일	PASS	06-인증-헤더-검증.png, 08-API-전체-테스트-통과.png

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T10	TOKEN_TTL=3, 발급 4 초 후 GET /me, POST, PUT, DELETE	401 TOKEN_EXPIRED, 화면 로그인 해제	401 / TOKEN_EXPIRED (4 건, check_api). 화면: 만료 안내 표시, 로그아웃 상태로 전환	PASS	09-토큰-만료-테스트-통과.png, 18-토큰-만료-화면.png
T11	로그인 → 로그아웃 → 내 정보 조회	401 TOKEN_REQUIRED, 헤더 없음	401, Authorization 헤더 없음, WWW-Authenticate: Bearer, 화면 로그아웃 상태 (같은 조건의 code 는 check_api 'no token me'에서 TOKEN_REQUIRED 확인)	PASS	16-로그아웃-내정보 조회-401.png
T12	Bob 토큰으로 글 등록 + authorId:1 / userId:1	400 BAD_INPUT, 글 미생성	400 / BAD_INPUT (2 건)	PASS	07-게시글-작성자-userId-조작-차단.png, 08-API-전체-테스트-통과.png
T13	Alice 토큰: 공백 제목, 공백 본문, 제목 101 자, 본문 5001 자	400 BAD_INPUT	400 / BAD_INPUT (4 건)	PASS	08-API-전체-테스트-통과.png
T14	없는 ID 9007199254740991 GET·PUT·DELETE(정상 토큰), GET /posts/abc	404 / 400	404 / NOT_FOUND (3 건), 400 / BAD_INPUT	PASS	08-API-전체-테스트-통과.png
T15	글 작성 → 백엔드 Ctrl+C → java -jar 재실행	글·작성자·내용 유지	200 / 해당 없음, 재시작 후 동일 글 표시	PASS	14-재시작-전.png, 15-재시작-후.png
T16	로그인 후 F5	글 유지, 로그인 해제	200 / 해당 없음, 글 2 건 유지, 화면 로그아웃 상태	PASS	17-새로고침-후-로그아웃.png
T17	토큰 없이 PUT·DELETE / Bob 작성·본인 수정·삭제	401 / 201·200·204	401 / TOKEN_REQUIRED (2 건) / 201, 200, 204	PASS	08-API-전체-테스트-통과.png

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	200 (목록·상세·로그인·/me·본인 수정)	03-로그인-응답.png, 08-API-전체-테스트-통과.png
새 글 등록	**201 및 Location**	201, Location: /api/posts/{id}가 응답 id 와 일치	13-글등록-Bearer-헤더.png, 08-API-전체-테스트-통과.png
본인 글 삭제	**204 및 빈 본문**	204, 본문 없음, 이후 GET 404	08-API-전체-테스트-통과.png
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	400 / BAD_INPUT (공백·길이 초과·authorId·userId·ID abc)	05-로그인-입력값-검증.png, 07-게시글-작성자-userId-조작-차단.png, 08-API-전체-테스트-통과.png
잘못된 계정	**401 INVALID_CREDENTIALS**	401 / INVALID_CREDENTIALS (틀린 비밀번호·없는 이메일)	05-로그인-입력값-검증.png, 08-API-전체-테스트-통과.png
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401 / TOKEN_REQUIRED (/me·POST·PUT·DELETE 무토큰)	06-인증-헤더-검증.png, 16-로그아웃-내정보 조회-401.png
변조·클레임 오류	**401 INVALID_TOKEN**	401 / INVALID_TOKEN (서명 변조 4 건). iss·aud 등 클레임 단독 오류는 별도 미시험	06-인증-헤더-검증.png, 08-API-전체-테스트-통과.png
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401 / TOKEN_EXPIRED (/me·POST·PUT·DELETE, TTL 3 초)	09-토큰-만료-테스트-통과.png, 18-토큰-만료-화면.png
다른 작성자의 변경	**403 FORBIDDEN**	403 / FORBIDDEN (Bob → Alice 글 PUT·DELETE)	08-API-전체-테스트-통과.png
없는 글	**404 NOT_FOUND**	404 / NOT_FOUND (GET·PUT·DELETE)	08-API-전체-테스트-통과.png

WWW-Authenticate: check_api.py 는 INVALID_CREDENTIALS 를 제외한 모든 401 응답에서 WWW-Authenticate 헤더가 bearer 로 시작하는지 검사하며, 401 에서 모두 통과했다. 서버에서는 제공 코드 Errors.api 가 상태 401 일 때 이 헤더를 붙인다.

401 과 403 화면: 401 을 받으면 api.js request()가 clearSession()을 호출하고 main.jsx run()이 setUser(getSession().user)로 null 을 반영해 로그아웃 상태로 바뀌며 만료·로그인 필요 안내가 뜬다. 403 은 session 을 지우지 않아 로그인 표시가 유지되고 권한 안내만 뜬다. 화면에서는 타인 글의 버튼이 숨겨져 403 을 직접 일으키기 어려워 PowerShell·check_api 로 서버 응답을 확인했다.

데이터 불변: check_api.py 의 'failed request leaves data unchanged' 검사가 Bob 의 PUT 거부, DELETE 거부, 변조 토큰 요청 뒤에 각각 공개 GET 으로 글 객체 전체를 다시 조회해 원본과 같은지 비교했고 3 번 모두 통과했다.

정상 토큰 + 없는 글: 정상 Alice 토큰으로 PUT/DELETE /api/posts/9007199254740991 → 404 / NOT_FOUND (check_api: missing protected PUT·DELETE).

테스트 증거

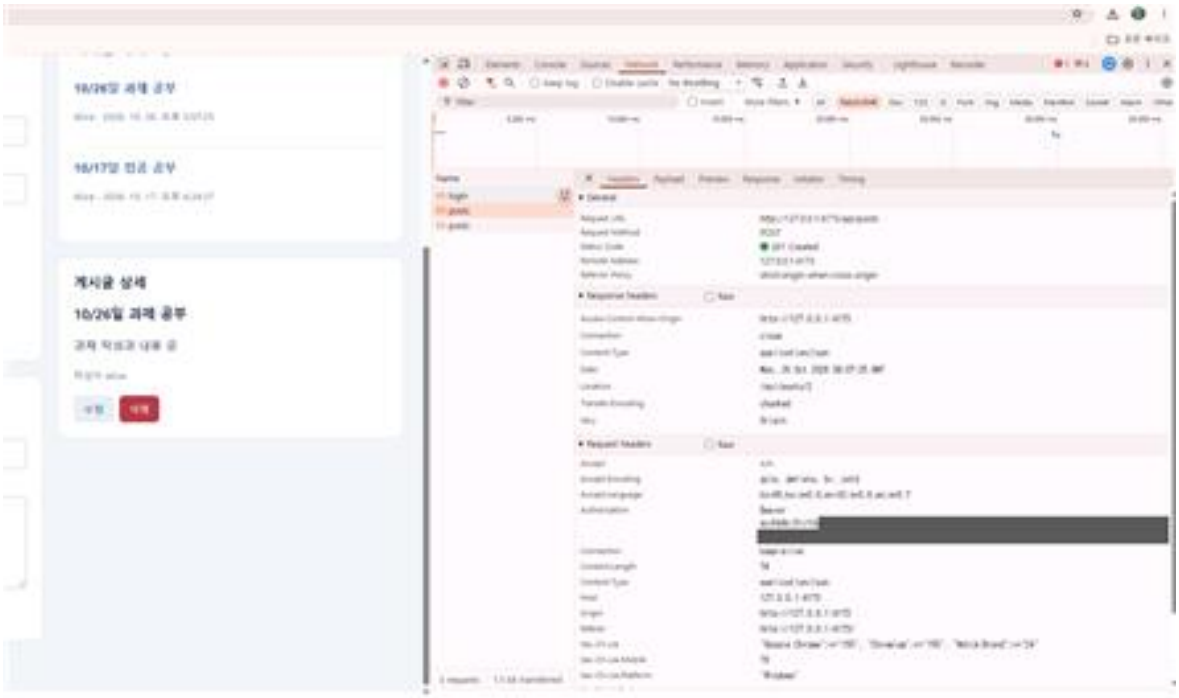
캡처에는 테스트 ID 와 실제 응답을 표시하세요. 토큰 원문과 서버 비밀 키는 가립니다.

로그인과 토큰 전송

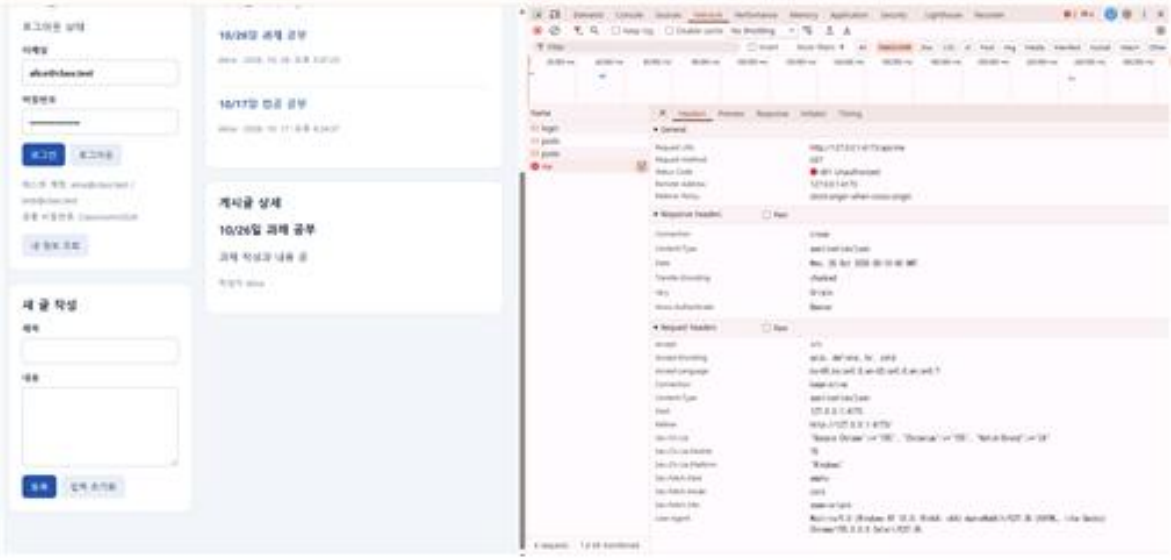
[증거 1: 로그인 성공과 토큰이 첨부된 요청. 토큰 원문은 가리세요.]



로그인 성공 200 응답: user(id·name·email), expiresIn 90 (토큰 원문은 가림)



빌드본에서 글 등록 201, Location 헤더, Authorization: Bearer 헤더 (토큰 뒷부분 가림)



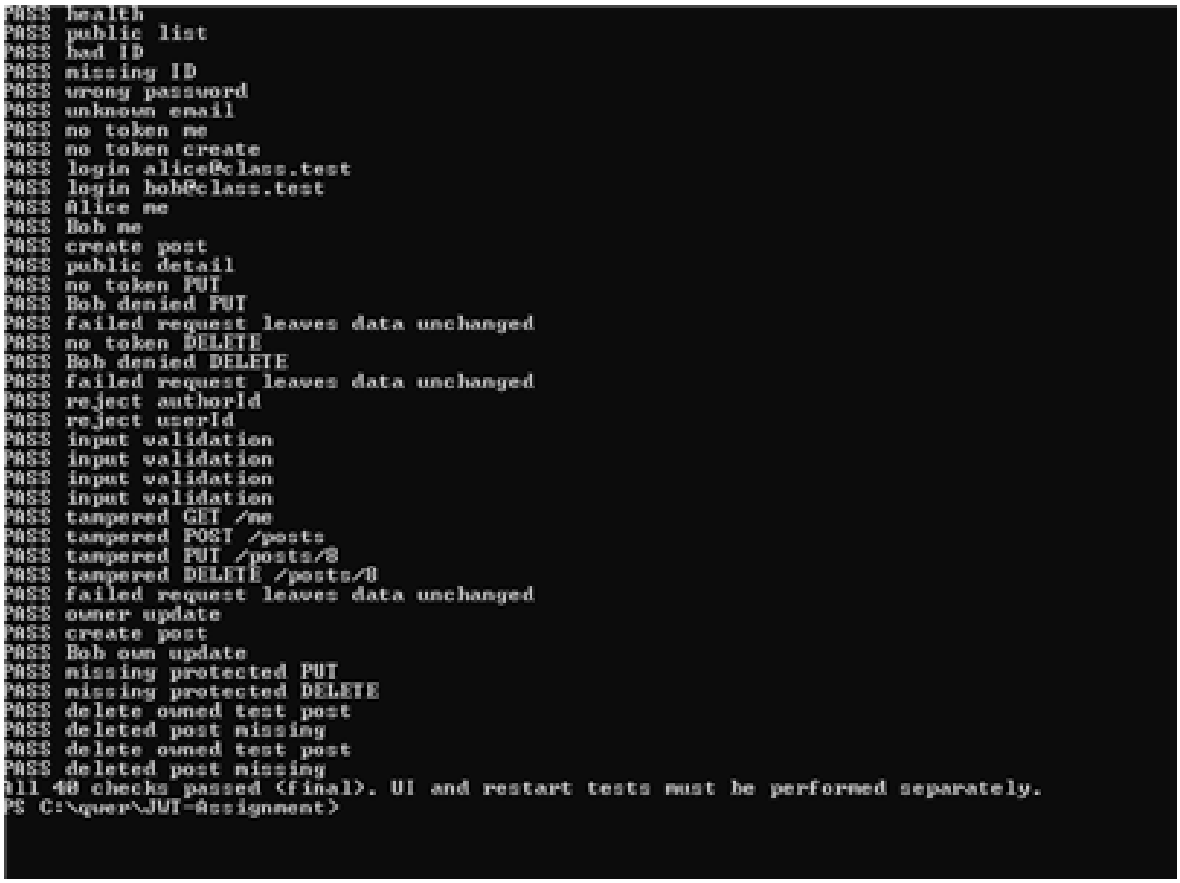
로그아웃 후 내 정보 조회: 401, Authorization 헤더 없음, WWW-Authenticate: Bearer



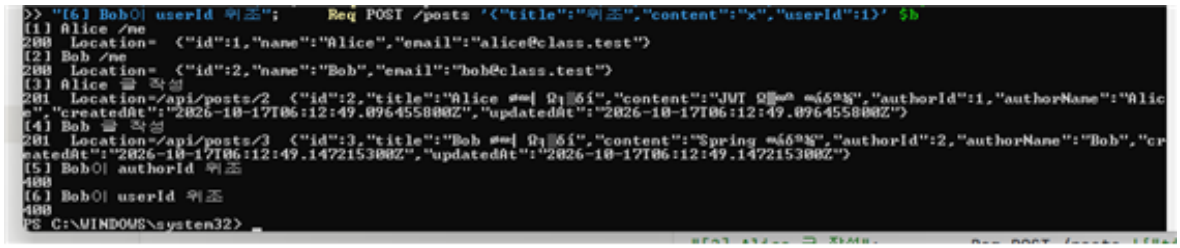
새로고침 후 글은 유지되고 로그인은 해제됨

다른 작성자의 수정과 삭제

[증거 2: Bob 이 Alice 의 글 수정·삭제를 시도한 서버 응답]



check_api --mode final: Bob denied PUT/DELETE(403)와 failed request leaves data unchanged 통과, All 40 checks passed —



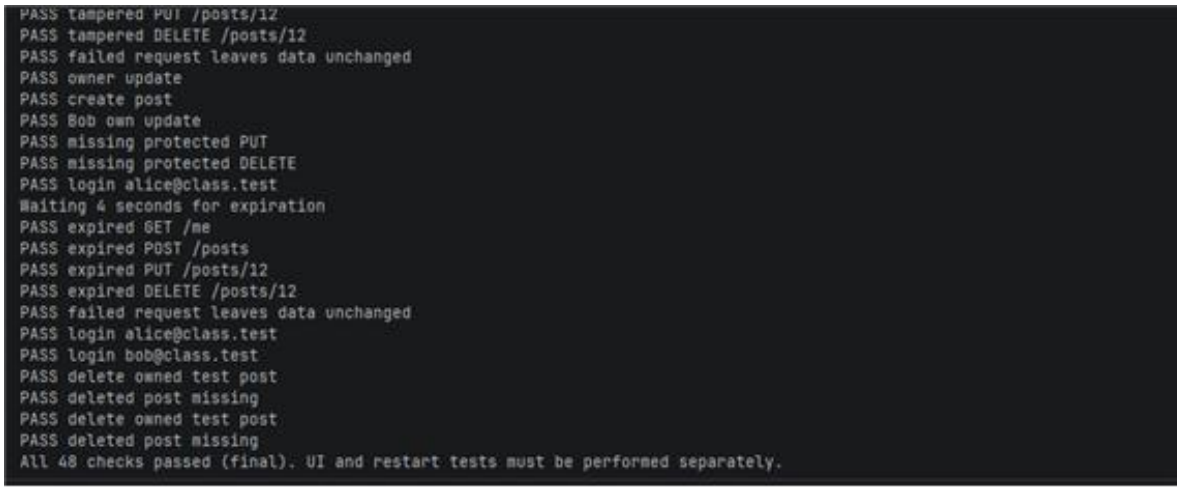
Alice·Bob 글 작성 201(authorId 1·2), 본문에 authorId·userId 를 넣은 위조 요청 400 — T05, T12

토큰 변조와 만료

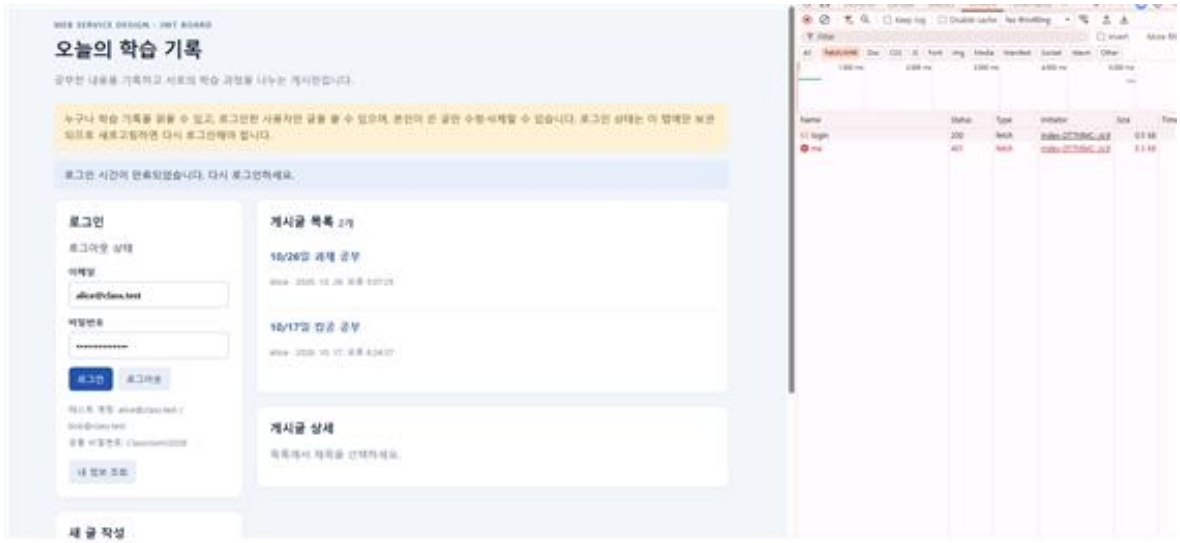
[증거 3: 토큰 변조·만료 시 서버 응답과 화면 처리]



. 헤더 없음·형식 오류 401 TOKEN_REQUIRED, 가짜·서명 변조 토큰 401 INVALID_TOKEN, 정상 토큰 200 — T04, T09



TOKEN_TTL=3 에서 만료 토큰으로 /me·POST·PUT·DELETE 401 TOKEN_EXPIRED, All 48 checks passed — T10



토큰 만료 후 화면: 만료 안내 표시와 로그아웃 상태 전환, me 401 — T10

자동 검사와 로컬 배포 증거

자동 검사: 저장소 루트에서 백엔드를 TOKEN_TTL=90 으로 실행하고 py tools/check_api.py --mode final 을 실행해 'All 40 checks passed (final)'을 확인했다(그림 08). 이어서 TOKEN_TTL=3 으로 재시작한 뒤 --mode final --expiry 를 실행해 'All 48 checks passed (final)'을 확인했다(그림 09). 이 도구는 상태 코드와 JSON code, 201 의 Location, 401 의 WWW-Authenticate, 거부 후 데이터 불변, 만료 토큰 4 건을 자동으로 확인한다. 화면 동작과 서버 재시작 유지는 이 도구가 확인하지 않으므로 따로 수행했다.

파일: docs/evidence/12-프론트엔드-빌드.png, 13-글등록-Bearer-헤더.png , 14-재시작-전.png / 15-재시작-후.png, 02-JWT 시크릿 오류로 실행 실패.png

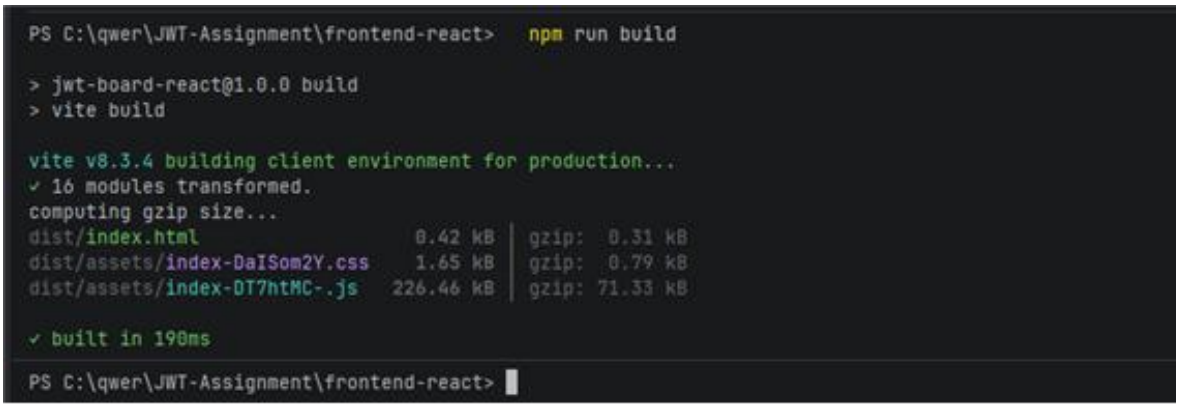


그림 12. npm run build 성공



그림 14. 백엔드 재시작 전 글 목록



그림 15. 백엔드 재시작 후 같은 글 유지 — T15



그림 02. 형식이 틀린 JWT_SECRET 으로 실행 시 서버 시작 거부 — BE-01-2

AI 활용과 참고 자료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: 사용

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
claude-opus-5-5	학습·설계·생성	AuthService.java 생성자·issueToken·currentUserId·bearerToken·assertOwner	초반 코드 기반으로 추가적인 코드 생성
claude-opus-5-5	생성	api.js authorizationHeaders·request·login·canEdit	초반 코드 기반으로 추가적인 코드 생성 및 틀린 코드 수정
claude-opus-5-5	디버깅·테스트	PowerShell 테스트 명령, 서버 실행 오류 원인 확인, check_api 실행 절차	현재까지 실행한 코드가 잘 돌아가고 있는지 검증
claude-opus-5-5	보고서	구조 와 변경 내역, 이해도 확인 테스트 증거	정확한 구조를 물어보고 보고서에 작성, 이해도 2,3 번을 설명해달라하고 이해한 것 바탕으로 정리 그림에 맞는 문구 추천

대표 사례

사례 1: 서버 JWT 검증(BE-02)

도구 / 날짜: Claude / 2026-10-17

요청한 내용: "BE-02 여기까지 완성했는데 뒷부분 구현이 어려워 단계별로 알려줘"처럼 과제 문장 한 항목씩 구현과 설명을 요청했다.

AI 가 제안한 내용: currentUserId 에서 헤더 추출을 bearerToken()으로 분리하고, Jwts.parser().verifyWith().requireIssuer().requireAudience()로 검증, ExpiredJwtException 을 먼저 catch 해 TOKEN_EXPIRED 와 INVALID_TOKEN 을 구분, HS256·exp 존재·sub 형식·계정 존재를 추가로 확인하는 코드를 제안했다.

실제 반영 위치: backend-spring/.../AuthService.java / currentUserId, bearerToken

내가 확인하고 수정한 내용: catch 순서가 바뀌면 만료가 INVALID_TOKEN 으로 처리되는 점, JJWT 가 exp 없는 토큰의 만료 검사를 건너뛰는 점을 설명받고 확인했다. 단계마다 예상 응답이 나와서 코드는 수정하지 않았다.

검증 방법과 결과: 단계별 PowerShell 요청: 'Bearer abc'와 서명 변조 토큰 → 401 INVALID_TOKEN, TTL=3 에서 5 초 후 → 401 TOKEN_EXPIRED. 최종 check_api --expiry 48 개 통과(T09, T10).

사례 2: 서버 실행 오류 디버깅

도구 / 날짜: Claude / 2026-10-17

요청한 내용: java -jar 실행 시 나온 오류 화면을 캡처해서 원인을 물었다.

AI 가 제안한 내용: 'JWT_SECRET 은 64 자리 16 진수여야 합니다'는 우리 생성자의 키 검사가 동작한 것이라고 설명하고, $env:JWT_SECRET = node -e ...로 키를 바로 넣고 Length 64 를 확인하게 했다. 이어서 'Port 8000 was already in use'는 IntelliJ 에서 켜 둔 서버 때문이라 정지하라고 했다. check_api --expiry 가 멈춘 원인은 IntelliJ 실행 설정에 TOKEN_TTL 이 없어서라고 짚었다.

실제 반영 위치: 실행 절차 , IntelliJ 실행 설정

내가 확인하고 수정한 내용: 키 형식 오류 화면은 형식 오류 키 거부의 증거로 남겼다. 코드는 바꾸지 않고 실행 방법만 고쳤다.

검증 방법과 결과: 'Started BoardApplication' 확인, check_api --expiry 48 개 통과, 재시작 후 데이터 유지(T15) 확인.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
제공 과제 문서 docs/01~07 (공지, API 명세, 체크리스트, 테스트 시나리오, README 양식, 로컬 실행 안내)	전체	요구사항··테스트 기준으로 사용
수업 제공 시작 코드 (backend-spring, frontend-react, tools/check_api.py)	BoardController, PostStore, Users, Errors 등	수정 없이 사용, 자동 검사에 사용
JJWT https://github.com/jwtk/jjwt	AuthService	builder·parser API 사용법 참고
Spring Boot 3.5 System Requirements https://docs.spring.io/spring-boot/3.5/system-requirements.html	실행 환경	JDK 버전 조건 확인
Vite Guide https://vite.dev/guide/	frontend-react	dev·build·preview 실행 방식 확인

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

Bob 이 글을 쓰면서 본문에 "authorId": 1 을 넣으면 Alice 이름으로 글이 저장되는 작성자 위조가 생긴다. 내 코드에서는 작성자를 본문이 아니라 AuthService.currentUserId()가 토큰을 검증해 돌려준 ID 로 저장한다. 또한 fields()가 title·content 외의 필드를 400 BAD_INPUT 으로 거부한다. T12 에서 Bob 토큰 + authorId:1, userId:1 요청이 모두 400 이었고 글이 생기지 않았다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼 숨기기(canEdit)는 화면에 그리지 않을 뿐이라 PowerShell 이나 curl 로 PUT·DELETE 요청을 직접 보내면 서버에 그대로 도착한다. 실제 권한 검사는 AuthService.assertOwner()가 DB 에 저장된 글의 authorId 와 currentUserId() 결과를 비교해 다르면 403 FORBIDDEN 을 던지며, DB 변경 전에 실행된다. T07·T08 에서 Bob 토큰으로 직접 보낸 PUT·DELETE 가 403 이었고 재조회한 글은 그대로였다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 JWT 의 가운데 부분을 Base64URL 로 풀어 읽기만 하는 것이라 누구나 할 수 있고, 내용을 바꿔도 알아채지 못한다. verify 는 서버만 아는 키로 서명을 다시 계산해 토큰의 서명과 비교하므로 한 글자만 바뀌어도 실패한다. 내 코드는 Jwts.parser().verifyWith(key)로 서명을, requireIssuer·requireAudience 로 iss·aud 를, 라이브러리로 만료를 검증하고, 추가로 알고리즘 HS256 고정, exp 존재, sub 형식, 실제 계정 존재를 확인한다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 clearSession()으로 이 탭 메모리의 토큰과 사용자를 지워 이후 요청에 헤더가 붙지 않게 한다(서버는 바뀌지 않음). 만료는 exp 가 지나 서버가 401 TOKEN_EXPIRED 를 돌려주는 것이고, 프론트는 이를 받아 session 을 비운다. 새로고침은 JavaScript 메모리가 초기화되어 로그인만 풀리고 글은 DB 에 남는다.
서버는 토큰 목록을 저장하지 않으므로 이미 복사된 토큰은 로그아웃해도 exp 까지는 유효하다. 그래서 TTL 을 90 초로 짧게 둔다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

가장 어려웠던 오류는 토큰 만료 자동 검사가 실패한 것이다. 앞의 검사 35 개는 모두 PASS 였는데, 마지막에 "FAIL Set TOKEN_TTL=3 before --expiry"가 나오며 멈췄고, 검사 도구가 만든 테스트 글(ID 10, 11)이 지워지지 않고 남았다.

처음에는 만료 처리 코드에 문제가 있다고 생각했다. 그런데 오류 메시지를 다시 읽어 보니 검사 도구가 로그인 응답의 expiresIn 이 10 초 이하인지 먼저 확인하고 있었고, 서버가 90 을 돌려주고 있었다. 즉 코드가 아니라 서버가 TTL 90 초로 실행되고 있는 것이 원인이었다. IntelliJ 실행 설정의 환경 변수 표를 열어 보니 JWT_SECRET 만 있고 TOKEN_TTL 항목이 없었다. 그래서 application.properties 의 기본값 ${TOKEN_TTL:90}이 적용되고 있었다. 환경 변수 표에 TOKEN_TTL=3 을 추가하고 서버를 재시작했다. 남은 테스트 글 10, 11 은 각각 Alice 와 Bob 으로 로그인해 삭제했다. 다시 --expiry 검사를 실행하자 4 초 대기 후 만료 토큰으로 보낸 /me·POST·PUT·DELETE 가 모두 401 TOKEN_EXPIRED 였고 "All 48 checks passed"가 나왔다(그림 09). 검사 후에는 TOKEN_TTL 을 90 으로 되돌렸다. 이 과정에서 Spring 이 .env 를 읽지 않아 환경 변수를 실행 설정이나 터미널에서 직접 넣어야 한다는 점과, 설정값이 없을 때 기본값이 조용히 쓰인다는 점을 확인했다.

현재 남은 문제와 다음에 개선할 점: 404 응답 시 안내 문구만 보이고 상세 화면 닫기·목록 자동 갱신은 하지 않는다. 다음에는 main.jsx 의 404 처리를 개선해야겠다.

제출 확인: [✓] 실제 코드와 설명 일치 [✓] 테스트 직접 수행 [✓] AI 활용 및 출처 기록 [✓] 비밀 키와 토큰 제거

GitHub 확인: [✓] 루트 README 작성 [✓] 보고서 PDF 와 증거 업로드 [✓] README 문서 링크 확인 [✓] 교수자 접근 가능 [✓] 저장소 URL 과 커밋 SHA 제출